

# CheddaBoards Unity SDK Setup Guide

Leaderboards, achievements, and auth for Unity. Any platform.

SDK version 2.2.7 • Docs: docs.cheddaboard.com • Dashboard: cheddaboard.com • Support: info@cheddaboard.com

## What's in this package

CheddaBoards is permanent, serverless gaming infrastructure powered by the Internet Computer. The SDK is a single C# file with zero dependencies — pure UnityWebRequest, no JavaScript bridges, no third-party packages. It works identically on Windows, Mac, Linux, iOS, Android, WebGL, consoles, and VR/AR.

|                                |                                                 |
|--------------------------------|-------------------------------------------------|
| CheddaBoards/                  |                                                 |
| - CheddaBoards.cs              | The SDK. This is the only file your game needs. |
| - README.md                    | Quick start and links                           |
| - LICENSE.md                   | MIT (SDK) + third-party font attribution        |
| - Demo/                        | CheddaClick - a complete working example game   |
| - CheddaClick.unity            | Demo scene                                      |
| - CheddaClickGame.cs           | Fully commented reference integration           |
| - ZeroCool.ttf                 | Demo display font (SIL OFL 1.1, GGBotNet)       |
| - ZeroCool SDF.asset           | TextMeshPro font asset for the demo UI          |
| - OFL.txt                      | The font's license                              |
| - Documentation/               |                                                 |
| - CheddaBoards-Setup-Guide.pdf | This guide                                      |

Minimum Unity version: 2022.3 LTS. No setup beyond the steps below — the SDK auto-creates its own singleton `GameObject` at runtime. The demo UI uses TextMeshPro; Unity prompts to import TMP Essentials (free, built in) when you open the scene.

## Quick start (3 minutes)

### 1. Get your keys

Create a free account at cheddaboard.com, register your game, and copy your API key and game ID from the dashboard.

### 2. Configure the SDK

```
var cb = CheddaBoards.Instance; // Auto-creates the singleton GameObject
cb.SetApiKey("your-api-key");
cb.SetGameId("your-game-id");
```

### 3. Log in and submit scores

```
void Start()
{
    var cb = CheddaBoards.Instance;
    cb.SetApiKey("your-api-key");
    cb.SetGameId("your-game-id");

    cb.OnLoginSuccess += (nickname) => Debug.Log($"Welcome {nickname}!");
    cb.OnScoreSubmitted += (score, streak) => Debug.Log($"Score saved: {score}");

    cb.LoginAnonymous(); // no name: returning players keep their saved nickname
}

void OnGameOver(int score, int streak)
{
    CheddaBoards.Instance.SubmitScore(score, streak);
}
```

That's the whole integration. Scores are stored permanently on-chain and served from the CheddaBoards canister, with automatic proxy fallback.

## Running the demo — CheddaClick

CheddaClick is a 30-second cheese-clicking game showing the full integration: anonymous login, guest flow, play sessions, score submit, leaderboard rendering, delta-synced achievements with an unlock panel, play count, and nickname changes.

Open Demo/CheddaClick.unity, select the object holding the CheddaClickGame component, enter your API key and game ID in the Inspector, and press Play.

CheddaClickGame.cs is commented as a reference. The two patterns worth copying into your own game are the event wiring and the anonymous-player ordering: load the profile before starting play, sync achievements after the score submit.

## Authentication

### Anonymous login

Instant login with a persistent device ID — no account creation needed.

```
cb.OnLoginSuccess += (nickname) => Debug.Log("Logged in: " + nickname);
cb.OnLoginFailed += (error) => Debug.Log("Failed: " + error);
cb.LoginAnonymous();
```

**Important:** log in without a name unless the player has just chosen one. Any name you pass becomes the current nickname and overwrites whatever the player had saved. Left empty, returning players keep their stored name and new players stay unnamed — render them as "Guest" until they pick a name via ChangeNickname().

Nickname rules (server-enforced): 3–16 characters; letters, numbers, and underscores. Taken names get a suffixed variant assigned by the server.

### Social login — Google / Apple / Internet Identity

Uses the Device Code flow (RFC 8628), so it works on every platform including consoles, VR, and native builds — no browser pop-ups in-game. The player visits cheddaboard.com/link and enters a short code (or scans the QR you display).

```
cb.OnDeviceCodeReceived += (code, url, qrDataUrl) =>
{
    // qrDataUrl is a base64 PNG data URL - show it as a scannable QR,
    // or just display the code and url as text.
    codeLabel.text = $"Go to {url}\nEnter code: {code}";
};
cb.OnDeviceCodeApproved += (nickname) => Debug.Log($"Welcome {nickname}!");
cb.OnDeviceCodeExpired += () => Debug.Log("Code expired, try again");

cb.LoginWithDeviceCode();
```

Sessions persist across restarts — device-code sign-in is a one-time flow. If a stored session is later rejected by the server, OnSessionExpired fires (along with OnLogoutSuccess).

### Account migration

Upgrade an anonymous player to a verified account without losing anything: scores and streaks merge by maximum, achievements are deduplicated, and play counts are summed — so linking the same account from a second device merges cleanly.

```
cb.OnAccountUpgraded += (oldProfile, newProfile) =>
    Debug.Log("Account upgraded! Scores preserved.");

cb.MigrateAnonymousToCurrent(anonymousDeviceId);
```

## Scores & leaderboards

### Submit a score

```
cb.OnScoreSubmitted += (score, streak) => Debug.Log($"Saved: {score}");
cb.OnScoreError += (error) => Debug.Log($"Error: {error}");

cb.SubmitScore(1500, 5); // score, streak
```

SubmitScore fans out to every standard board on your game (all-time, weekly, daily).

### Targeted boards (per-level / per-mode)

For category leaderboards, submit to one board by ID. This writes to that board only — no fan-out, and it doesn't touch the player's overall profile total. The board must be configured as targeted in the dashboard.

```
cb.OnScoreSubmittedToBoard += (boardId, score, streak) =>
    Debug.Log($"Saved to {boardId}: {score}");

cb.SubmitScoreToBoard("level-14", 1500, 5);
```

## Read scoreboards

Every game gets three standard boards created automatically: all-time, weekly, and daily — those exact strings are their board IDs. SubmitScore fans out to all of them. You can also create extra boards (monthly, seasonal, per-mode) at any reset interval in your dashboard.

```
cb.OnScoreboardLoaded += (id, config, entries) =>
{
    foreach (Dictionary<string, object> entry in entries)
        Debug.Log($"#{entry["rank"]} {entry["nickname"]}: {entry["score"]}");
};

cb.GetWeeklyLeaderboard();
cb.GetDailyLeaderboard();
cb.GetAlltimeLeaderboard();
cb.GetMonthlyLeaderboard(); // requires a monthly board created in your dashboard

// Or by ID - works for any board, timed or targeted (default limit 100)
cb.GetScoreboard("weekly", 100);
cb.GetScoreboard("all-time", 100);
cb.GetScoreboard("level-14", 100);
```

## Player rank and archives

```
cb.OnScoreboardRankLoaded += (id, rank, score, streak, total) =>
    Debug.Log($"You are #{rank} of {total} players");

cb.GetScoreboardRank("weekly");

// Previous periods (last week's results, etc.)
cb.OnArchivedScoreboardLoaded += (archiveId, config, entries) =>
    Debug.Log($"Archive {config["periodStart"]} - {config["periodEnd"]}");

cb.GetLastWeekScoreboard();
cb.GetLastMonthScoreboard(); // requires a monthly board created in your dashboard
cb.GetYesterdayScoreboard();
```

## Achievements

```
cb.OnAchievementUnlocked += (id) => Debug.Log($"Unlocked: {id}");

cb.UnlockAchievement("first_win"); // single
cb.UnlockAchievementsBatch(new List<string>{"first_win", "speed_run"}); // batch

// Read the player's achievements (served from the profile)
cb.OnAchievementsLoaded += (achievements) =>
    Debug.Log($"Got {achievements.Count} achievements");
cb.GetAchievements();

// Or attach achievements to a score submit - they sync automatically
// after the score is confirmed:
cb.SubmitScoreWithAchievements(2000, 10,
    new List<string>{"first_win", "high_scorer", "streak_5"});
```

## Play sessions (anti-cheat)

Server-side time validation ensures scores match actual play time. Start a session when gameplay begins; the session token is attached automatically to both SubmitScore and SubmitScoreToBoard.

```
cb.OnPlaySessionStarted += (token) => Debug.Log("Session started");

cb.StartPlaySession(); // when gameplay begins
cb.SubmitScore(score, streak); // token attached automatically
cb.EndPlaySession(); // when the player quits or pauses
```

## Events reference

| Event                      | Parameters                                       | Fires when                        |
|----------------------------|--------------------------------------------------|-----------------------------------|
| OnSdkReady                 | –                                                | SDK initialised                   |
| OnLoginSuccess             | nickname                                         | Login completed                   |
| OnLoginFailed              | error                                            | Login failed                      |
| OnLogoutSuccess            | –                                                | Logged out                        |
| OnSessionExpired           | –                                                | Stored session rejected (401/403) |
| OnScoreSubmitted           | score, streak                                    | Score saved (fan-out)             |
| OnScoreSubmittedToBoard    | boardId, score, streak                           | Targeted score saved              |
| OnScoreError               | error                                            | Score submission failed           |
| OnScoreboardLoaded         | id, config, entries                              | Scoreboard data received          |
| OnScoreboardRankLoaded     | id, rank, score, streak, total                   | Player rank received              |
| OnAchievementUnlocked      | achievementId                                    | Achievement unlocked              |
| OnAchievementsLoaded       | achievements                                     | Achievement list received         |
| OnPlaySessionStarted       | token                                            | Play session active               |
| OnDeviceCodeReceived       | code, url, qrDataUrl                             | Device code ready to display      |
| OnDeviceCodeApproved       | nickname                                         | Social login completed            |
| OnDeviceCodeExpired        | –                                                | Code timed out                    |
| OnAccountUpgraded          | oldProfile, newProfile                           | Migration completed               |
| OnProfileLoaded            | nickname, score, streak, achievements, playCount | Profile data received             |
| OnNicknameChanged          | nickname                                         | Nickname updated                  |
| OnArchivesListLoaded       | scoreboardId, archives                           | Archive list received             |
| OnArchivedScoreboardLoaded | archiveId, config, entries                       | Archived scoreboard data          |

## Utility methods

```
cb.IsAuthenticated() // true if logged in
cb.HasAccount()      // true if logged in with a non-anonymous account
cb.IsAnonymous()     // true if using anonymous auth
cb.CanConnect()      // true if API key or session is set
cb.GetNickname()     // current nickname ("" for unnamed anonymous - show "Guest")
cb.GetHighScore()    // cached high score
cb.GetBestStreak()   // cached best streak
cb.GetPlayCount()    // cached play count
cb.GetPlayerId()     // persistent device ID
```

## Configuration

| Property     | Default | Description                                         |
|--------------|---------|-----------------------------------------------------|
| apiKey       | –       | Your CheddaBoards API key                           |
| gameId       | –       | Your game ID                                        |
| debugLogging | false   | Verbose console logging (set true while developing) |

The SDK auto-creates a singleton `GameObject` with `DontDestroyOnLoad`. No manual scene setup is required.

## Support & links

Full guides and the complete REST API reference: [docs.cheddaboards.com](https://docs.cheddaboards.com)

Dashboard and account: [cheddaboards.com](https://cheddaboards.com)

Questions and support: [info@cheddaboards.com](mailto:info@cheddaboards.com)

Unity SDK repository: [github.com/cheddatech/CheddaBoards-Unity](https://github.com/cheddatech/CheddaBoards-Unity)  
Open-source backend: [github.com/cheddatech/cheddaboards](https://github.com/cheddatech/cheddaboards)  
Godot SDK: [github.com/cheddatech/CheddaBoards-Godot](https://github.com/cheddatech/CheddaBoards-Godot)

CheddaBoards Unity SDK 2.2.7 — © CheddaTech Ltd. MIT licensed. Demo font Zero Cool © GGBotNet, SIL OFL 1.1 (see LICENSE.md and Demo/OFL.txt).